Lab 11：OpenCV 行人偵測實驗報告

Raspberry Pi / Python / Image Processing / Pedestrian Detection

學號	614530036	**姓名**	邱世宇
Email	jasonchiu0916@gmail.com	**實驗平台**	Raspberry Pi OS / Python venv
課程主題	OpenCV 影像處理與行人偵測	**報告格式**	依 Lab9_HW 範本整理

一、實驗名稱

OpenCV 行人偵測專題：在 Raspberry Pi 上進行影像處理、影片處理、攝影機讀取與行人偵測。

二、實驗目的

- 完成 Raspberry Pi OS 環境設定，並透過 SSH / SFTP 進行程式與影像檔案操作。
- 建立 Python 虛擬環境，安裝 OpenCV、imutils、NumPy 等影像處理套件。
- 練習 OpenCV 的基本操作：讀圖、縮放、ROI 擷取、畫框、灰階轉換與 CLAHE 影像增強。
- 使用 HOGDescriptor 與 People Detector 偵測圖片中的行人，並將偵測結果輸出成 result.jpg。
- 測試影片與攝影機輸入，理解 cv2.VideoCapture、CascadeClassifier 與輸出影片的基本流程。

三、實驗步驟

1. 環境準備

使用 Raspberry Pi Imager 安裝 Raspberry Pi OS，設定主機名稱、帳號密碼、Wi-Fi 與 SSH。開機後可用 ssh jason@JasonPi 或指定 IP 連線。

2. 建立與啟用 Python 環境

在 lab11 資料夾建立虛擬環境並啟用，例如 source ~/env/bin/activate；執行程式時命令列會出現 (env)。

3. 安裝必要套件

安裝 OpenCV、imutils、numpy 等套件，並使用 python -c "import cv2; print(cv2.__version__)" 確認 OpenCV 能正常匯入。

4. 測試圖片讀取與 ASCII Art

執行 test_opencv.py，讀取 img/Lenna.png，將圖片縮小並轉為 ASCII 字元輸出到終端機。

5. 測試基本影像處理

執行 processing.py，完成圖片 resize、ROI 擷取、rectangle 畫框、gray 灰階、CLAHE 影像增強，並輸出 clahe.jpg。

6. 完成 sample.py 填空

將 img/pedestrian.jpg 或 img/input.jpg 讀入，先依比例縮放為寬 400，再使用 HOG 偵測行人並把偵測目標框起來。

7. 測試影片處理

執行 video_processing.py，讀取 video/video.mp4，透過分類器偵測目標並輸出 video/output.avi；若使用 SSH，需註解 cv2.imshow() 與 cv2.waitKey()。

8. 測試攝影機輸入

執行 webcam.py 時，使用 cv2.VideoCapture(0) 讀取 webcam 即時畫面，偵測到目標後以綠框標示。

四、實驗 Demo 與問題討論

Demo 1-1：完成 sample.py 的圖片縮放填空

題目要求將原始圖片依比例縮放，寬度固定為 400，不能直接把高寬都寫死，否則圖片比例會變形。關鍵程式碼如下：

```
image = cv2.imread("img/pedestrian.jpg")

# 以寬度 400 為基準，保持原圖比例縮放
image = imutils.resize(image, width=400)
```

Demo 1-2：完成 sample.py 的偵測目標畫框填空

HOG 偵測後會取得每個目標的左上與右下座標，使用 cv2.rectangle() 將所有最終保留下來的偵測框畫在圖片上。關鍵程式碼如下：

```
for (xA, yA, xB, yB) in pick:
    cv2.rectangle(image, (xA, yA), (xB, yB), (0, 255, 0), 2)

cv2.imwrite("result.jpg", image, [cv2.IMWRITE_JPEG_QUALITY, 98])
```

完整程式碼 sample.py

```
from __future__ import print_function

from imutils.object_detection import non_max_suppression

from imutils import paths

import numpy as np

import argparse

import imutils

import cv2


# construct the argument parse and parse the arguments
```

```
# Create a HOGDescriptor object
hog = cv2.HOGDescriptor()

# Initialize the People Detector
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())



image = cv2.imread("img/input.jpg")

# rescale origin image to 400 (以寬度為基準，保持比例)(使用 imutils 函式進行 rescale)
# ======
image = imutils.resize(image, width=400)
# ======

orig = image.copy()
cv2.imwrite("result.jpg", image, [cv2.IMWRITE_JPEG_QUALITY, 98])

# detect people in the image
(rects, weights) = hog.detectMultiScale(
    image,
    winStride=(4, 4),
    padding=(8, 8),
    scale=1.05
)

# draw the original bounding boxes
for (x, y, w, h) in rects:
    cv2.rectangle(orig, (x, y), (x + w, y + h), (0, 0, 255), 2)

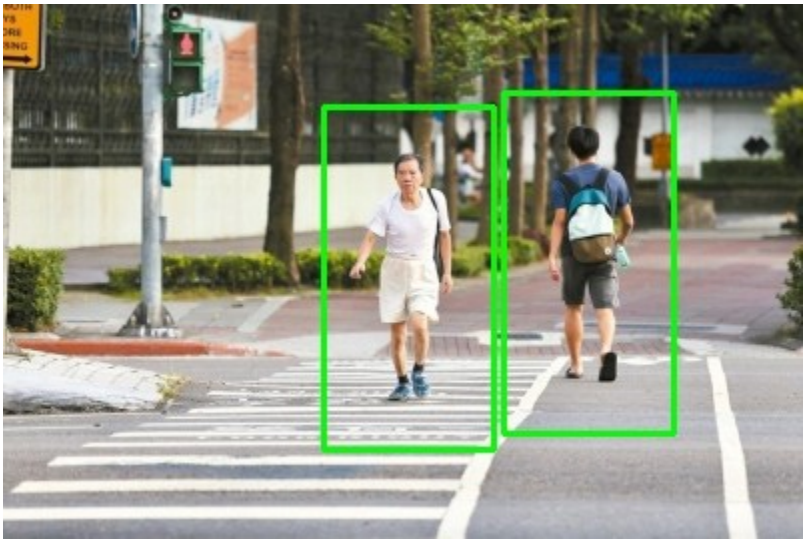
# apply non-maxima suppression to the bounding boxes using a
# fairly large overlap threshold to try to maintain overlapping
# boxes that are still people
rects = np.array([[x, y, x + w, y + h] for (x, y, w, h) in rects])
pick = non_max_suppression(rects, probs=None, overlapThresh=0.65)

# draw the final bounding boxes
```

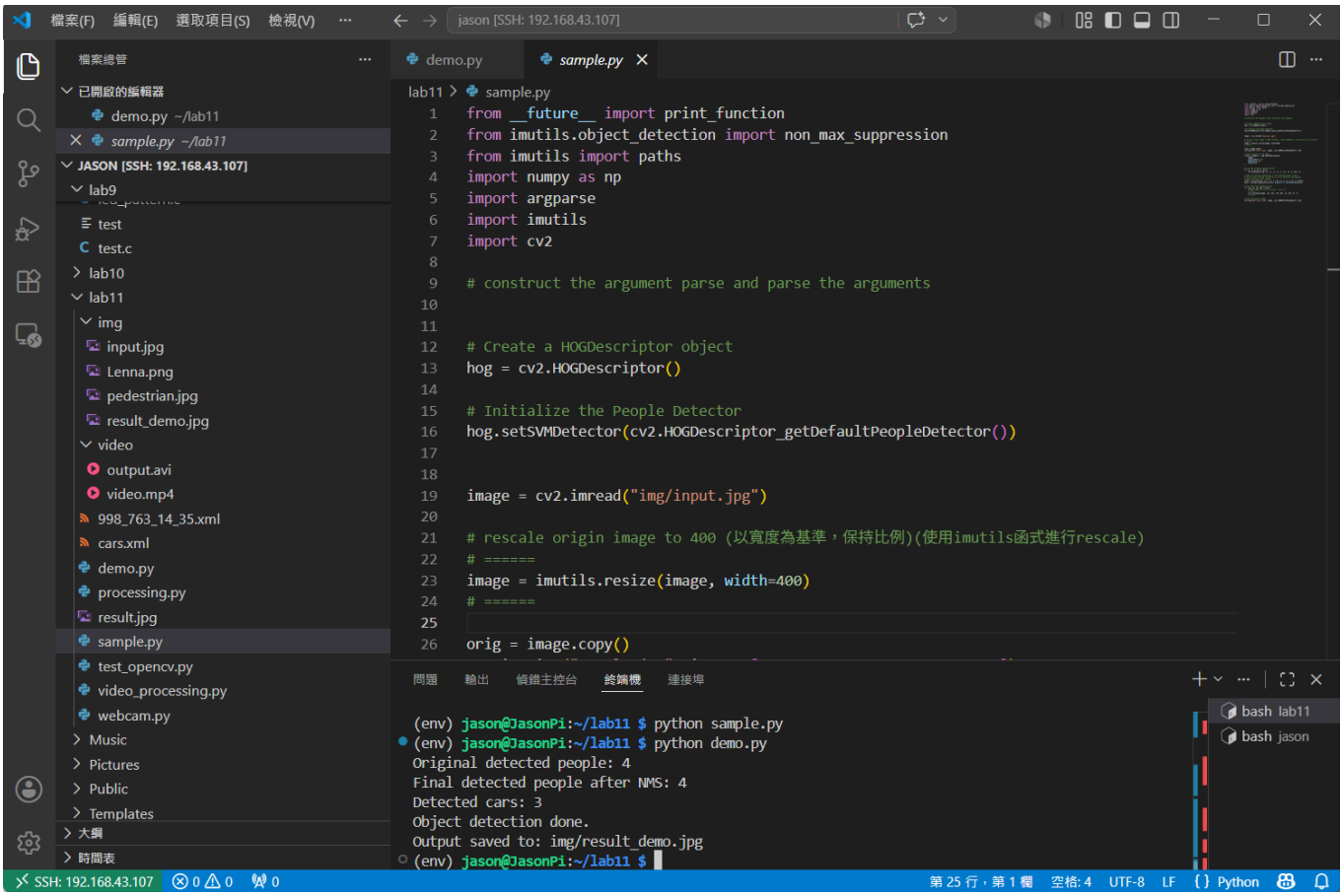
```
for (xA, yA, xB, yB) in pick:
    # 得到行人的四個點，在原始圖案中將其括出來
    # ======
    cv2.rectangle(image, (xA, yA), (xB, yB), (0, 255, 0), 2)
    # ======


# show the output images
cv2.imwrite("result.jpg", image, [cv2.IMWRITE_JPEG_QUALITY, 98])
```

result.jpg



透過 VS Code Remote SSH 連到樹莓派



Demo 2：程式功能簡單說明

- sample.py 先建立 HOGDescriptor，並載入 OpenCV 內建的 DefaultPeopleDetector。
- 程式讀取圖片後，先將圖片寬度縮放為 400，降低運算量並保持長寬比例。
- hog.detectMultiScale() 會在不同尺度的影像中尋找可能是行人的區域。
- non_max_suppression() 用來合併重疊框，避免同一個人被框出很多次。
- 最後用 cv2.rectangle() 將行人框起來，並輸出 result.jpg 作為 demo 結果。

Demo 3：延伸想法

本次延伸應用是將原本的行人偵測擴充成行人與車輛偵測。程式中使用 OpenCV 內建的 HOG + SVM 方法偵測行人，並另外使用 cars.xml 作為車輛偵測模型，讓輸出結果可以同時標示出畫面中的 Person 與 Car。

這個功能可以應用在簡單的停車場或道路監控系統。例如 Raspberry Pi 接上攝影機後，可以用來觀察畫面中是否有人或車輛出現，並進一步搭配截圖、存檔或時間紀錄，做成一個簡易的智慧監控 。

demo.py

```
from __future__ import print_function

from imutils.object_detection import non_max_suppression

import numpy as np

import imutils
```

```
import cv2
import os
# ========================
# 檔案路徑設定
# ========================

IMAGE_PATH = "img/pedestrian.jpg"
OUTPUT_PATH = "img/result_demo.jpg"
CAR_XML_PATH = "cars.xml"
# ========================
# 工具函式
# ========================

def check_file(path, error_message):
    if not os.path.exists(path):
        print(error_message)
        exit(1)



def load_image(image_path, width=400):
    image = cv2.imread(image_path)

    if image is None:
        print(f"Error: 圖片讀取失敗：{image_path}")
        exit(1)

    # 照 Lab11 sample.py：以寬度 400 為基準縮放
    image = imutils.resize(image, width=width)
    return image

def create_people_detector():
    hog = cv2.HOGDescriptor()
    hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
    return hog
```

```
def create_car_detector(car_xml_path):
    car_classifier = cv2.CascadeClassifier(car_xml_path)

    if car_classifier.empty():
        print("Error: 車子偵測 XML 讀取失敗")
        exit(1)

    return car_classifier
# =========================
# 偵測函式
# =========================

def detect_people(image, hog):
    rects, weights = hog.detectMultiScale(
        image,
        winStride=(4, 4),
        padding=(8, 8),
        scale=1.05
    )

    print(f"Original detected people: {len(rects)}")

    rects = np.array([[x, y, x + w, y + h] for (x, y, w, h) in rects])
    people_pick = non_max_suppression(rects, probs=None, overlapThresh=0.65)

    print(f"Final detected people after NMS: {len(people_pick)}")

    return people_pick



def detect_cars(image, car_classifier):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    cars = car_classifier.detectMultiScale(
        gray,
        scaleFactor=1.1,
```

```
        minNeighbors=6,
        minSize=(50, 50)
    )

    print(f"Detected cars: {len(cars)}")

    return cars
# ========================
# 繪圖函式
# ========================

def draw_people(image, people):
    for (xA, yA, xB, yB) in people:
        cv2.rectangle(
            image,
            (xA, yA),
            (xB, yB),
            (0, 255, 0),
            2
        )

        cv2.putText(
            image,
            "Person",
            (xA, max(yA - 10, 20)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 0),
            2
        )
def draw_cars(image, cars):
    for (x, y, w, h) in cars:
        cv2.rectangle(
            image,
            (x, y),
            (x + w, y + h),
```

```
            (255, 0, 0),
            2
        )

        cv2.putText(
            image,
            "Car",
            (x, max(y - 10, 20)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (255, 0, 0),
            2
        )



# =========================
# 主程式
# =========================

def main():
    check_file(IMAGE_PATH, f"Error: 找不到圖片：{IMAGE_PATH}")
    check_file(CAR_XML_PATH, f"Error: 找不到車子偵測模型：{CAR_XML_PATH}")

    image = load_image(IMAGE_PATH)

    hog = create_people_detector()
    car_classifier = create_car_detector(CAR_XML_PATH)

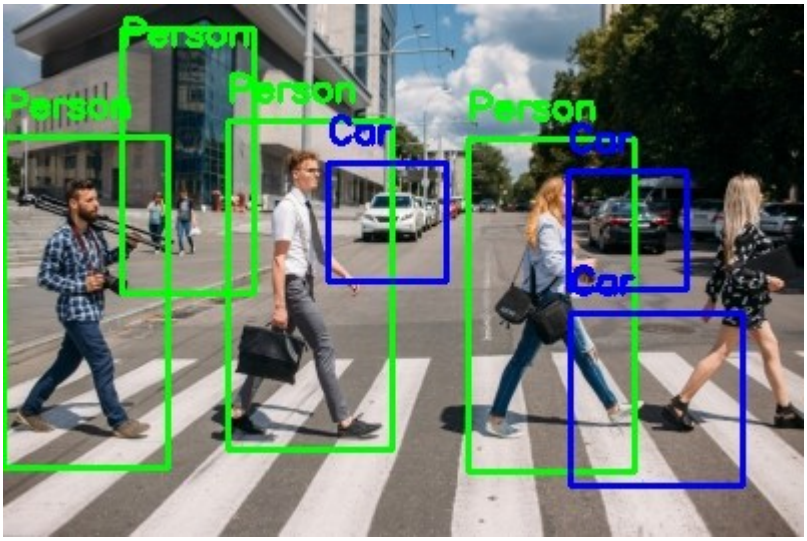
    people = detect_people(image, hog)
    cars = detect_cars(image, car_classifier)

    draw_people(image, people)
    draw_cars(image, cars)

    cv2.imwrite(OUTPUT_PATH, image, [cv2.IMWRITE_JPEG_QUALITY, 98])
```

```
    print("Object detection done.")
    print(f"Output saved to: {OUTPUT_PATH}")
if __name__ == "__main__":
    main()
```

result_demo.jpg



Demo 4：心得

這次 Lab11 一開始最容易卡住的地方不是 OpenCV 本身，而是檔案路徑。例如 input.jpg 放在 img 資料夾裡，但程式卻寫成 input.jpg，就會造成 cv2.imread() 讀不到圖片，後面 resize 就會出現 NoneType 錯誤。

另外，用 SSH 執行影片程式時，cv2.imshow() 也會出問題，因為 SSH 終端機沒有圖形介面可以開視窗。所以後來我改成註解 imshow，只輸出 output.avi，這樣比較符合 Raspberry Pi 遠端操作的情境。

透過這次實驗，我比較清楚知道 OpenCV 的基本流程其實就是「讀取影像、前處理、偵測、畫出結果、輸出檔案」。雖然程式碼不算很長，但每一個檔案路徑、模型檔和顯示環境都要對，整個 demo 才會順利完成。